INDEX

S.NO	CONTENT	PG.NO	SIGNATURE
1.	WRITE A PROGRAM TO CREATE A TABLE WITH THE FOLLOWING PARAMETERS - SNO, SNAME, DOB, SUB1, SUB2 AND SUB3 USING APPROPRIATE DATA TYPE.	4	
2.	WRITE A QUERY TO INSERT DATA INTO TABLE AND DISPLAY THE TABLE.”	4	
3.	WRITE A QUERY TO DISPLAY THE DATA IN THE TABLE AS FOLLOWS ● SELECTED ROWS AND COLUMNS. ● SELECTED COLUMNS AND ALL ROWS. ● ALL ROWS AND ALL COLUMNS. SELECTED ROWS AND ALL COLUMNS.	5	
4.	WRITE A QUERY TO ELIMINATE DUPLICATE ROWS USING SELECT STATEMENTS.	6	
5.	WRITE A QUERY TO SORT THE DATA IN A TABLE.	6	
6.	WRITE A QUERY TO CREATE A TABLE FROM ANOTHER TABLE.	6	
7.	WRITE A QUERY TO INSERT DATA INTO A TABLE FROM ANOTHER TABLE.	6,7	
8.	WRITE A QUERY TO DELETE ALL ROWS FROM THE TABLE.	7	
9.	WRITE A QUERY TO MODIFY THE EXISTING COLUMNS IN A TABLE.	7	
10.	WRITE A QUERY TO TRUNCATE A TABLE.	7	
11.	WRITE A QUERY TO DESTROY A TABLE.	7,8	
12.	WRITE A QUERY TO CREATE A TABLE CUSTOMER WITH CUSTID AS PRIMARY KEY FOLLOWED BY OTHER COLUMNS.	8	
13.	WRITE A QUERY TO ESTABLISH FOREIGN KEY CONSTRAINTS.	8	
14.	WRITE A QUERY TO CALCULATE THE ANNUAL SALARY OF AN EMPLOYEE.	8	
15.	WRITE A QUERY TO CREATE A TABLE HAVING BNAME & BNO, DISPLAY VALUES WHERE BNO IS NULL.	8	

16.	WRITE A QUERY TO CREATE A TABLE THAT DOES TAKE NULL VALUES.	8,9	
17.	WRITE A QUERY TO IMPLEMENT CONSTRAINTS.	9	
18.	WRITE A QUERY TO DROP PRIMARY KEY CONSTRAINT FROM A TABLE.	9	
19.	WRITE A QUERY TO IMPLEMENT OR.	9	
20.	WRITE A QUERY TO IMPLEMENT NOT.	9,10	
21.	WRITE A QUERY TO IMPLEMENT RANGE SEARCHING.	10	
22.	WRITE A QUERY TO DISPLAY THE EMPLOYEE NAME THAT BEGINS WITH R.	10	
23.	WRITE A QUERY TO IMPLEMENT THE SYSTEM DATE.	10,11	
24.	WRITE A QUERY TO FIND THE AVERAGE SALARY OF AN EMP TABLE.	11	
25.	WRITE A QUERY TO THE MINIMUM VALUE OF SALARY IN THE EMP TABLE.	11	
26.	WRITE A QUERY TO THE MAXIMUM VALUE OF SALARY IN THE EMP TABLE.	11	
27.	WRITE A QUERY TO IMPLEMENT A COUNT FUNCTION IN THE EMP TABLE.	11	
28.	WRITE A QUERY TO IMPLEMENT A SUM FUNCTION IN THE EMP TABLE.	11	

PL/SQL QUERIES

S.NO	CONTENT	PG.NO	SIGNATURE
1.	PL/SQL TO ADD TWO NUMBERS.	12	
2.	PL/SQL FOR DEMO ON 'FOR' LOOP	12	
3.	PL/SQL TO SWAP TWO NUMBERS.	12,13	
4.	PL/SQL TO PRINT EMPNO, ENAME, JOB AND SALARY OF AN EMPLOYEE GIVEN EMPNO	13	
5.	PL/SQL TO IMPLEMENT %FOUND.	14,15	
6.	PL/SQL TO IMPLEMENT %FOUND USING DEPT TABLE.	15	
7.	PL/SQL USING CURSOR TO DISPLAY DETAILS OF ALL EMPLOYEES FROM EMP TABLE WHOSE SUM OF SAL AND COMM IS GREATER THAN 2000.	15,16	
8.	PL/SQL BLOCK THAT RETRIEVES COURSE TABLE INTO CURSOR. INPUT CODE ID TO SEARCH. IF A COURSE EXISTS PRINT THE INFORMATION. IF COURSE ID DOESN'T EXIST, PRINT USER DEFINED MESSAGE BY THROWING USER DEFINED EXCEPTION.	16,17	
9.	PL/SQL TO HANDLE ZERO_DIVIDE EXCEPTION.	17,18	
10.	PL/SQL THAT CREATES A TRIGGER THAT INSERTS OR UPDATES THE VALUES OF ENAME.JOB AS LOWER CASE STRING.	17	
11.	PL/SQL FOR SIMPLE LOOP.	18	

STRUCTURED QUERY LANGUAGE

1) WRITE A QUERY TO CREATE A TABLE WITH THE FOLLOWING PARAMETERS SNO,SNAME,DOB,SUB1,SUB2,AND SUB3 USING A APPROPRIATE DATA TYPE.

SQL> CREATE TABLE STUD1(SNO NUMBER(11)UNIQUE,SNAME VARCHAR(13) NOT NULL,DOB DATE,SUB1 CHAR(8),SUB2 CHAR(8),SUB3 CHAR(8));

TABLE CREATED.

SQL> DESC STUD1

```
Table created.

SQL> desc stud1
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 SNO                                                NUMBER(11)
 SNAME                                     NOT NULL VARCHAR2(13)
 DOB                                                DATE
 SUB1                                               CHAR(8)
 SUB2                                               CHAR(8)
 SUB3                                               CHAR(8)
```

2) WRITE A QUERY TO INSERT DATA INTO TABLE AND DISPLAY THE TABLE.

SQL> INSERT INTO STUD1 VALUES(&SNO,'&SNAME','&DOB','&SUB1','2 &SUB2','&SUB3');

```
SQL> insert into stud1 values(&sno,'&sname','&dob','&sub1','2 &sub2','&sub3');
Enter value for sno: 1
Enter value for sname: sailaja
Enter value for dob: 3-oct-1994
Enter value for sub1: Eng
Enter value for sub2: Litera
Enter value for sub3: Phylo
old   1: insert into stud1 values(&sno,'&sname','&dob','&sub1','2 &sub2','&sub3')
new   1: insert into stud1 values(1,'sailaja','3-oct-1994','Eng','2 Litera','Phylo')

1 row created.

SQL> /
Enter value for sno: 2
Enter value for sname: shanmukh
Enter value for dob: 11-mar-1999
Enter value for sub1: phy
Enter value for sub2: ldc
Enter value for sub3: m1
old   1: insert into stud1 values(&sno,'&sname','&dob','&sub1','2 &sub2','&sub3')
new   1: insert into stud1 values(2,'shanmukh','11-mar-1999','phy','2 ldc','m1')

1 row created.
SQL> /
Enter value for sno: 3
Enter value for sname: hardik jain
Enter value for dob: 02-oct-2005
Enter value for sub1: engg
Enter value for sub2: m2
Enter value for sub3: pis
old   1: insert into stud1 values(&sno,'&sname','&dob','&sub1','2 &sub2','&sub3')
new   1: insert into stud1 values(3,'hardik jain','02-oct-2005','engg','2 m2','pis')

1 row created.
```

SQL> SELECT * FROM STUD1;

```
SQL> select * from stud1;

       SNO SNAME         DOB       SUB1     SUB2     SUB3
---------- ------------- --------- -------- -------- --------
         1 sailaja       03-OCT-94 Eng      2 Litera Phylo
         2 shanmukh      11-MAR-99 phy      2 ldc    m1
         3 hardik jain   02-OCT-05 engg     2 m2     pis
```

3) WRITE A QUERY TO DISPLAY THE DATA IN TABLE AS I. SELECT ROWS AND COLUMNS II. SELECTED COLUMNS AND ALL ROWS ALL ROWS III. ALL ROWS AND ALL COLUMNS IV. SELECTED ROWS AND ALL COLUMNS

I. SQL> SELECT * FROM STUD1 WHERE SNO>2;

```
SQL> select * from stud1 where sno>2;

       SNO SNAME         DOB       SUB1     SUB2     SUB3
---------- ------------- --------- -------- -------- --------
         3 hardik jain   02-OCT-05 engg     2 m2     pis
```

II. SQL> SELECT SNO,SNAME FROM STUD1;

```
SQL> select sno,sname from stud1;

       SNO SNAME
---------- -------------
         1 sailaja
         2 shanmukh
         3 hardik jain
```

III. SQL> SELECT *FROM STUD1;

```
SQL> select *from stud1;

       SNO SNAME         DOB       SUB1     SUB2     SUB3
---------- ------------- --------- -------- -------- --------
         1 sailaja       03-OCT-94 Eng      2 Litera Phylo
         2 shanmukh      11-MAR-99 phy      2 ldc    m1
         3 hardik jain   02-OCT-05 engg     2 m2     pis
```

IV. SQL> SELECT SNO,SNAME FROM STUD1 WHERE SNO=1;

```
SQL> select sno,sname from stud1 where sno=1;

       SNO SNAME
---------- -------------
         1 sailaja
```

4) WRITE A QUERY TO ELIMINATE DUPLICATE ROWS USING SELECT STATEMENT.
SQL> SELECT DISTINCT SUB1 FROM STUD1;

```
SQL> select distinct sub1 from stud1;

SUB1
--------
engg
Eng
phy
```

5) WRITE A QUERY TO SORT THE DATA IN A TABLE.
SQL> SELECT *FROM STUD1 ORDER BY SNO DESC;

```
SQL> select *from stud1 order by sno desc;

       SNO SNAME         DOB       SUB1     SUB2     SUB3
---------- ------------- --------- -------- -------- --------
         3 hardik jain   02-OCT-05 engg     2 m2     pis
         2 shanmukh      11-MAR-99 phy      2 ldc    m1
         1 sailaja       03-OCT-94 Eng      2 Litera Phylo
```

6) SELECT A QUERY TO CREATE TABLE FROM ANOTHER TABLE.

SQL> CREATE TABLE STUD4 AS SELECT *FROM STUD1;

TABLE CREATED.
SQL> SELECT * FROM STUD4;

```
SQL> select * from stud4;

       SNO SNAME         DOB       SUB1     SUB2     SUB3
---------- ------------- --------- -------- -------- --------
         1 sailaja       03-OCT-94 Eng      2 Litera Phylo
         2 shanmukh      11-MAR-99 phy      2 ldc    m1
         3 hardik jain   02-OCT-05 engg     2 m2     pis
```

7) WRITE A QUERY TO INSERT DATA INTO A TABLE FROM ANOTHER TABLE.
SQL> CREATE TABLE STUD6(SNO NUMBER(11)UNIQUE, SNAME VARCHAR(13) NOT NULL,DOB DATE,SUB1 CHAR(8),SUB2 CHAR(8),SUB3 CHAR(8));

TABLE CREATED.

SQL> INSERT INTO STUD6 SELECT * FROM STUD1 WHERE SNO<=2;

2 ROWS CREATED.

SQL> SELECT * FROM STUD6;

```
SQL> select * from stud6;

       SNO SNAME         DOB       SUB1     SUB2     SUB3
---------- ------------- --------- -------- -------- --------
         1 sailaja       03-OCT-94 Eng      2 Litera Phylo
         2 shanmukh      11-MAR-99 phy      2 ldc    m1
```

8) WRITE A QUERY TO DELETE ALL ROWS FROM A TABLE.
SQL> DELETE FROM STUD4;

```
SQL> delete from stud4;

4 rows deleted.
```

9) WRITE A QUERY TO MODIFY THE EXISTING COLUMNS IN A TABLE.
SQL> SELECT SNAME AS NAME FROM STUD1;

```
SQL> select sname as name from stud1;

NAME
--------------------
sailaja
B Samaresh
BS
```

10) WRITE A QUERY TO TRUNCATE A TABLE

SQL> TRUNCATE TABLE STUD1;

TABLE
TRUNCATED.

11) WRITE A QUERY TO DESTROY A TABLE.
SQL> DROP TABLE STUD1;

TABLE
DROPPED. 12)
CREATE A
TABLE CLIENT
MASTER
WHICH STORES
THE CLIENT
INFORMATION
SQL> CREATE
TABLE
CLIENT_MAS(C
NO
NUMBER(2),CN
AME

CHAR(20),CADD
VARCHAR2(9));
TABLE CREATED.
SQL> DESC CLIENT_MAS;

```
SQL> desc client_mas;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CNO                                                NUMBER(2)
 CNAME                                              CHAR(20)
 CADD                                               VARCHAR2(9)
```

12) WRITE A QUERY TO CREATE A TABLE CUSTOMER WITH CUSTID AS PRIMARY FOLLOWED BY OTHER COLUMNS.

SQL> CREATE TABLE CUSTOMER (CUSTID NUMBER(4) CONSTRAINT PKL PRIMARY KEY,PRODUCT VARCHAR2(5));

TABLE CREATED.

13) WRITE A QUERY TO ESTABLISH FOREIGN KEY CONSTRAINT.

SQL> CREATE TABLE PRODUCER (PRODUCT_ID NUMBER (8) UNIQUE,CUST_DEMAND NUMBER(4) REFERENCES CUSTOMER(CUSTID));

TABLE CREATED.

14) WRITE A QUERY TO CALCULATE THE ANNUAL SALARY OF AN EMPLOYEE

SQL>SELECT EMPNO, SAL*12 FROM EMP;

```
SQL> select empno,sal*12 from emp;

     EMPNO     SAL*12
---------- ----------
      7839      60000
      7698      34200
      7782      29400
      7566      35700
      7788      36000
      7902      36000
      7369       9600
      7499      19200
      7521      15000
      7654      15000
      7844      18000

     EMPNO     SAL*12
---------- ----------
      7876      13200
      7900      11400
      7934      15600

14 rows selected.
```

15) WRITE A QUERY TO CREATE A TABLE HAVING BNAME AND BNO, AND DISPLAY VALUE WHERE BNO IS NULL.

SQL> CREATE TABLE DEMO1(BNO NUMBER(10),BNAME CHAR(20));

TABLE CREATED.

SQL> INSERT INTO DEMO1 VALUES(1,'MADHU');
1 ROW CREATED.

SQL> INSERT INTO DEMO1 VALUES(NULL,'SINDHU');
1 ROW CREATED.

SQL> INSERT INTO DEMO1 VALUES(0,'HEMA');
1 ROW CREATED.

SQL> SELECT * FROM DEMO1 WHERE BNO IS NULL;

```
SQL> select * from demo1 where bno is null;

       BNO BNAME
---------- --------------------
           sindhu
```

16) WRITE A QUERY TO CREATE A TABLE THAT DOESNOT TAKE NULL VALUE.
SQL> CREATE TABLE DEMO2(NO NUMBER(10),NAME CHAR(20) NOT NULL); TABLE CREATED.

SQL> INSERT INTO DEMO2 VALUES(1,NULL);

```
SQL> insert into demo2 values(1,null);
insert into demo2 values(1,null)
                          *
ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."DEMO2"."NAME")
```

17) WRITE A QUERY TO IMPLEMENT CONSTRAINTS.
SQL> CREATE TABLE STU(SNO NUMBER(10) PRIMARY KEY,SNAME CHAR(30) UNIQUE,PHNO NUMBER(10)CHECK(PHNO>800000000),ADDRESS CHAR(10)NOT NULL);
TABLE CREATED.

SQL> DESC STU;

```
SQL> desc stu;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 SNO                                       NOT NULL NUMBER(10)
 SNAME                                              CHAR(30)
 PHNO                                               NUMBER(10)
 ADDRESS                                   NOT NULL CHAR(10)
```

18) WRITE A QUERY TO DROP PRIMARY KEY CONSTRAINT FROM A TABLE.
SQL> ALTER TABLE STU DROP PRIMARY KEY;
TABLE ALTERED.

19) WRITE A QUERY TO IMPLEMENT OR.

SQL> SELECT * FROM EMP WHERE ENAME='FORD' OR SAL=3000;

```
SQL> SELECT * FROM EMP WHERE eNAME='FORD' OR SAL=3000;

     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7902 FORD       ANALYST         7566 03-DEC-81       3000
        20
```

20) WRITE A QUERY TO IMPLEMENT NOT SQL> SELECT * FROM EMP WHERE NO SAL>2400;

```
SQL> select * from emp where not SAL>2400;

     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7369 SMITH      CLERK           7902 17-DEC-80        800
        20

      7499 ALLEN      SALESMAN        7698 20-FEB-81       1600        300
        30

      7521 WARD       SALESMAN        7698 22-FEB-81       1250        500
        30


     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7654 MARTIN     SALESMAN        7698 28-SEP-81       1250       1400
        30

      7844 TURNER     SALESMAN        7698 08-SEP-81       1500          0
        30

      7876 ADAMS      CLERK           7788 23-MAY-87       1100
        20


     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7900 JAMES      CLERK           7698 03-DEC-81        950
        30

      7934 MILLER     CLERK           7782 23-JAN-82       1300
        10
```

21) WRITE A QUERY TO IMPLEMENT RANGE SEARCHING.

SQL> SELECT * FROM EMP WHERE SAL BETWEEN 2000 AND 3000;

```
SQL> select * from emp where sal between 2000 and 3000;

     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7698 BLAKE      MANAGER         7839 01-MAY-81       2850
        30

      7782 CLARK      MANAGER         7839 09-JUN-81       2450
        10

      7566 JONES      MANAGER         7839 02-APR-81       2975
        20


     EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM
---------- ---------- --------- ---------- --------- ---------- ----------
    DEPTNO
----------
      7788 SCOTT      ANALYST         7566 19-APR-87       3000
        20

      7902 FORD       ANALYST         7566 03-DEC-81       3000
        20
```

22) WRITE A QUERY TO DISPLAY THE EMPLOYEE NAME THAT BEGINS WITH Q. SQL>
SELECT * FROM EMP WHERE ENAME LIKE '%Q';
NO ROWS
SELECTED.

23) WRITE A QUERY TO IMPLEMENT SYSTEM DATE.

SQL> SELECT SYSDATE FROM DUEAL;

```
SQL> SELECT SYSDATE FROM DUAL

SYSDATE
---------
02-OCT-24
```

24) WRITE A QUERY TO FIND THE AVERAGE SALARY OF EMP TABLE

SQL> SELECT AVG(SAL) “AVERAGE SALARY” FROM EMP;

```
SQL> SELECT AVG(SAL) “AVERAGE SALARY” FROM EMP;

AVERAGE SALARY
--------------
    2073.21429
```

25) WRITE A QUERY TO FIND THE MINIMUM VALUE OF SALARY OF EMP TABLE.

SQL> SELECT MIN(SAL) “MINIMUM SALARY” FROM EMP;

```
SQL> SELECT MIN(SAL) “MINIMUM SALARY” FROM EMP;

MINIMUM SALARY
--------------
           800
```

26) WRITE A QUERY TO FIND THE MAXIMUM VALUE OF SALARY OF EMP TABLE.

SQL> SELECT MAX(SAL) “MAXIMUM SALARY” FROM EMP;

```
SQL> SELECT MAX(SAL) “MAXIMUM SALARY” FROM EMP;

MAXIMUM SALARY
--------------
          5000
```

27) WRITE A QUERY TO IMPLEMENT COUNT FUNCTION ON EMP TABLE.

SQL>SELECT COUNT(*) “TOTAL NO OF ROWS” FROM EMP;

```
SQL> SELECT COUNT(*) “TOTAL NO OF ROWS” FROM EMP;

TOTAL NO OF ROWS
----------------
              14
```

28) WRITE A QUERY TO IMPLEMENT SUM FUNCTION ON TABLE.

SQL> SELECT SUM(SAL)”SUM OF SLARY” FROM EMP;

```
SQL> SELECT SUM(SAL)”SUM OF SLARY” FROM EMP;

SUM OF SLARY
------------
       29025
```

PL/SQL QUERIES

1. ADD 2 NUMBERS.

```
SQL> SET SERVEROUTPUT ON;
DECLARE
VAR1 INTEGER;
VAR2 INTEGER;
VAR3 INTEGER;
BEGIN
VAR1 := &VAR1;
VAR2 := &VAR2;
VAR3 := VAR1 + VAR2;
DBMS_OUTPUT.PUT_LINE('THE SUM OF ' || VAR1 || ' AND ' || VAR2 || '  IS: ' || VAR3);
END;  /
```

OUTPUT:

```
Enter value for var1: 10
old   6:   VAR1 := &VAR1;
new   6:   VAR1 := 10;
Enter value for var2: 20
old   7:   VAR2 := &VAR2;
new   7:   VAR2 := 20;
The sum of 10 and 20 is: 30

PL/SQL procedure successfully completed.
```

2. FOR LOOP DEMO

```
SQL> SET SERVEROUTPUT ON;
DECALARE
I NUMBER(2);
BEGIN
FOR I IN 1..10
LOOP
DBMS_OUTPUT.PUT_LINE(I);
END LOOP;
END;
```

OUTPUT:

```
1
2
3
4
5
6
7
8
9
10

PL/SQL procedure successfully completed.
```

3. PL/SQL PROGRAM TO SWAP TWO NUMBERS

(A) USING TEMP

```
SQL> SET SERVEROUTPUT ON;

DECLARE
```

```
A  NUMBER(2):=&A;
B  NUMBER(2):=&B;
TEMP NUMBER(2);
BEGIN
DBMS_OUTPUT.PUT_LINE('BEFORE SWAPPING     A='||A||'B='||B);
  TEMP:=A;
B:=TEMP;
DBMS_OUTPUT.PUT_LINE('AFTER SWAPPING A='||A||'B='||B);
END;
/
```

OUTPUT:

```
Enter value for a: 10
old   2:   A NUMBER(2):=&A;
new   2:   A NUMBER(2):=10;
Enter value for b: 20
old   3:   B NUMBER(2):=&B;
new   3:   B NUMBER(2):=20;
BEFORE SWAPPING A=10B=20
AFTER SWAPPING A=10B=10

PL/SQL procedure successfully completed.
```

(B) WITHOUT USING TEMP

```
SQL> SET SERVEROUTPUT ON;
 DECLARE
A  NUMBER (2):=&A;
B  NUMBER (2):=&B;
 BEGIN
 DBMS_OUTPUT.PUT_LINE('BEFORE SWAPPING’A='||A||'B='||B);
A:=A+B;
B:=A-B;
A:=A-B;
DBMS_OUTPUT.PUT_LINE('AFTER SWAPPING’A='||A||'B='||B);
END;
/
```

OUTPUT:

```
Enter value for a: 4
old   2:   A NUMBER(2) := &A;
new   2:   A NUMBER(2) := 4;
Enter value for b: 5
old   3:   B NUMBER(2) := &B;
new   3:   B NUMBER(2) := 5;
BEFORE SWAPPING A=4 B=5
AFTER SWAPPING A=5 B=4

PL/SQL procedure successfully completed.
```

4 . WRITE A PROGRAM TO PRINT EMPNO, ENAME,JOB,SALARY OF AN EMPLOYEE OF GIVEN EMPNO.

```
SQL> SET SERVEROUTPUT ON;
DECLARE
TYPE EMPREC IS RECORD (ENO   EMP.EMPNO%TYPE,N
EMP.ENAME%TYPE,J EMP.JOB%TYPE,S EMP.SAL%TYPE);
R EMPREC;
ENO EMP.EMPNO%TYPE := &ENO;
BEGIN
SELECT EMPNO, ENAME, JOB, SAL
INTO R
FROM EMP
WHERE EMPNO = ENO; -- Corrected the condition
DBMS_OUTPUT.PUT_LINE(R.ENO || ' ' || R.N || ' ' || R.J || ' '
|| R.S);
END;
/
```

OUTPUT:

```
Enter value for eno: 7839
old   4: ENO EMP.EMPNO%TYPE := &ENO;
new   4: ENO EMP.EMPNO%TYPE := 7839;
7839 KING PRESIDENT 5000

PL/SQL procedure successfully completed.
```

5 . PL/SQL PROGRAM TO IMPLEMENT %FOUND

```
SQL> SET SERVEROUTPUT ON;
DECLARE
CURSOR EMP_CUR IS
SELECT ENAME, COMM FROM
EMP
WHERE EMPNO = 7654
AND COMM IS NOT NULL;
EMP_NAME VARCHAR2(20); EMP_COMM
NUMBER(10,2);
BEGIN
OPEN EMP_CUR;
```

```
FETCH EMP_CUR INTO EMP_NAME, EMP_COMM;
IF EMP_CUR%FOUND THEN
DBMS_OUTPUT.PUT_LINE('EMPLOYEE: ' || EMP_NAME || ' GETS
COMMISSION: ' || EMP_COMM);
ELSE
DBMS_OUTPUT.PUT_LINE('EMPLOYEE DID NOT GET ANY COMMISSION');
END IF;
DBMS_OUTPUT.PUT_LINE(EMP_CUR%ROWCOUNT || ' RECORDS FETCHED SO  FAR');
CLOSE EMP_CUR;
END;
/
```

OUTPUT:

```
EMPLOYEE: MARTIN GETS COMMISSION: 1400
1 RECORDS FETCHED SO FAR

PL/SQL procedure successfully completed.
```

6. PL/SQL TO PMPLEMENT %FOUND USING DEPT TABLE.

```
SQL> SET SERVEROUTPUT ON;
DECLARE
CURSOR DEPTCUR IS SELECT DEPTNO, DNAME FROM DEPT WHERE DEPTNO =
10;
TDEPTCUR DEPTCUR%ROWTYPE;
BEGIN
OPEN DEPTCUR;
FETCH DEPTCUR INTO TDEPTCUR;
IF DEPTCUR%FOUND THEN
DBMS_OUTPUT.PUT_LINE('DEPTNO: ' || TDEPTCUR.DEPTNO || '  DNAME: ' ||
TDEPTCUR.DNAME);
ELSE
DBMS_OUTPUT.PUT_LINE('No department found with DEPTNO 10');
END IF;
CLOSE DEPTCUR;
END;
/
```

OUTPUT:

```
DEPTNO: 10  DNAME: ACCOUNTING

PL/SQL procedure successfully completed.
```

7. PL/SQL BLOCK USING THE COURSOR TO DISPLAY DETAILS OF ALL EMPLOYEE'S FROM EMP TABLE WHOSE SUM OF SAL AND COMM IS >2000

```
SQL> SET SERVEROUTPUT ON;
DECLARE
MEM_PNO EMP.EMPNO%TYPE;
MEM_NAME EMP.ENAME%TYPE;
MEM_SAL EMP.SAL%TYPE;
CURSOR C1 IS
SELECT EMPNO, ENAME, SAL + NVL(COMM, 0)
FROM EMP
WHERE SAL + NVL(COMM, 0) > 2000;
BEGIN
OPEN C1;
DBMS_OUTPUT.PUT_LINE('EMPNO  ENAME  SAL');
LOOP
FETCH C1 INTO MEM_PNO, MEM_NAME, MEM_SAL;  EXIT
WHEN C1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(MEM_PNO || '  ' || MEM_NAME || '  ' ||
        MEM_SAL);
END LOOP;
CLOSE C1;
END;
/
```

OUTPUT:

```
EMPNO  ENAME   MSAL
7839  KING  5000
7698  BLAKE  2850
7782  CLARK  2450
7566  JONES  2975
7902  FORD  3000
7654  MARTIN  2650

PL/SQL procedure successfully completed.
```

8. PL/SQL BLOCK THE RETRIVES COURSE TABLE INTO CURSOR.INPUT CODE ID TO SEARCH. IF COURSE EXIST PRINT THE INFORMATION. IF COURSE ID DOES NOT EXIST, PRINT USER DEFINED MESSAGE BY THROWING USER DEFINED EXCEPTION.

```
SQL> SET SERVEROUTPUT ON;
DECLARE
```

```
C_NAME COURSE.COURSENAME%TYPE;
COURSOR C1 IS SELECT COURSENAME FROM COURSE WHERE
COURSEIS:='&ID';
COURSE_NOT_FOUND EXCEPTION;
BEGIN
OPEN C1;
FETCH C1 INTO C_NAME;
IF C1% FOUND THEN DBMS_OUTPUT.PUT_LINE('COURSE
NAME IS'||C_NAME);
ELSE
RAISE COURSE_NOT_FOUND;
END IF;
EXCEPTION WHEN COURSE_NOT_FOUND THEN
DBMS_OUTPUT.PUT_LINE('COURSE ID NOT FOUND');
END;
/
```

OUTPUT:

```
Enter value for id: 13
old 3: cursorcl is select coursename from course where courseis=&id'
new 3:cursor cl is select coursename from course where courseis='13';
course name isB.Sc.computeres
PL/SQL procedure successfully completed.
SQL>/
Enter value for id: 18
old 3: cursor cl is select coursename from course where courseis=&id';
new 3:cursor c1 is select coursename from course where courseis=18';
course id is not found
PL/SQL procedure successfully completed
```

9. PL/SQL PROGRAM TO HANDLE ZERO DIVIDE EXCEPTION

```
SQL> SET SERVEROUTPUT ON;
DECLARE
X  NUMBER := &X;
Y  NUMBER := &Y;
BEGIN
IF Y <> 0 THEN
DBMS_OUTPUT.PUT_LINE('DIVISION=' || X / Y);
ELSE
DBMS_OUTPUT.PUT_LINE('CHECK THE DENOMINATOR');
END IF;
END;  /
```

OUTPUT:

```
Enter value for x: 23
old   2:    X NUMBER := &X;
new   2:    X NUMBER := 23;
Enter value for y: 9
old   3:    Y NUMBER := &Y;
new   3:    Y NUMBER := 9;
DIVISION=2.555555555555555555555555555555555555556

PL/SQL procedure successfully completed.
```

10. PL/SQL THAT CREATES A TRIGGER THAT INSERTS OR UPDATES THE VALUES OF ENAME.JOB AS LOWER-CASE STRING

```
SQL> CREATE OR REPLACE TRIGGER LOWERNAME
BEFORE INSERT OR UPDATE ON EMP FOR EACH ROW
BEGIN
:NEW.ENAME:=:LOWER(“NEW.ENAME);
NEW.JOB:=LOWER(:MEW.JOB);
END;
/
```

OUTPUT:

Trigger created.

SQL> UPDATEE EMP SET JOB=’MANAGER’ WHERE ENAME=SMITH;

```
SQL>  UPDATE EMP SET JOB=’MANAGER’ WHERE ENAME='SMITH';

1 row updated.
```

11. PL/SQL FOR SIMPLE LOOP.

```
SQL> SET SERVEROUTPUT ON;
DECLARE
I NUMBER:=0;
BEGIN
LOOP
DBMS_OUTPUT.PUT_LINE('I='||I);
I:=I+2;
EXIT WHEN I>10;
END LOOP;
END;
/
```

OUTPUT:

```
I=0
I=2
I=4
I=6
I=8
I=10

PL/SQL procedure successfully completed.
```